# Teste de streaming em Arduino/ESP32 antes × depois do refinamento

Como configurar e executar `examples/arduino_stream_test.py` — do pacote de teste no seu PC até o relatório gerado com a placa física

## 1. O que este teste mede

O pyruleanalyzer refina as regras de um modelo de árvores (Decision Tree, Random Forest ou GBDT) e exporta o resultado para um sketch Arduino. Este teste responde, **na placa real**, se o modelo refinado continua detectando anomalias tão bem quanto o original — e quanto ele economiza em memória e tempo de inferência.

O PC faz o papel dos sensores: envia **linhas reais do conjunto de teste** (nunca usadas no treino nem no refinamento), uma a cada `--period-ms`. O Arduino classifica cada amostra e dispara um **alarme** (e acende o `LED_BUILTIN`) depois de `--confirm` classificações anormais seguidas. O mesmo cenário é tocado duas vezes: com o sketch **before** (regras iniciais) e com o sketch **after** (regras finais).

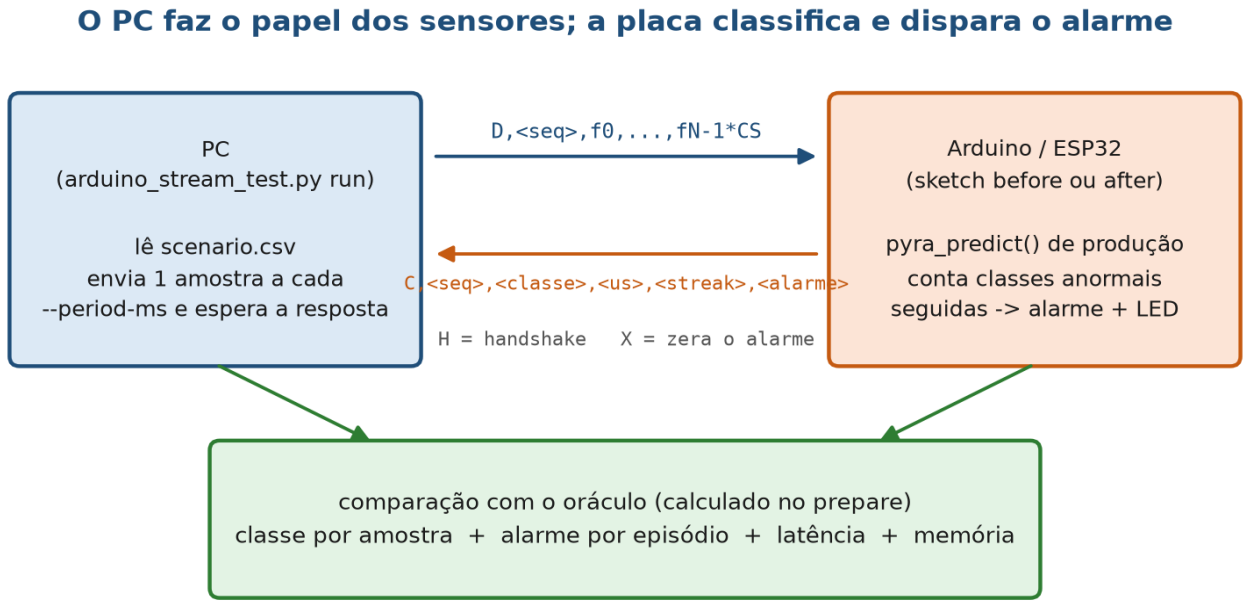



Figura 1 — o laço PC / placa. Como a lógica do alarme é determinística, o `prepare` já sabe em qual amostra cada variante deve alarmar (o "oráculo"); o `run` confere a placa contra ele.

## 2. Três comandos, duas pessoas

O script é dividido para que quem tem a placa precise apenas de uma pasta (o **pacote**), sem a base de dados de treino nem o pyruleanalyzer instalado:

| Comando | Quem roda | O que faz |
|---|---|---|
| `prepare` | Você, no seu PC (sem placa) | Treina ou carrega o modelo, exporta os dois sketches, monta o cenário com linhas reais de teste, calcula o oráculo e escreve o pacote. |
| `run` | Quem tem a placa | Grava cada sketch, toca o cenário, registra cada amostra e escreve `results/<execução>/`. |

| Comando | Quem roda | O que faz |
|---|---|---|
| `report` | Qualquer um | Reimprime a comparação before × after de uma execução (o `run` já faz isso no final). |

# 3. Configuração (uma vez só)

## 3.1 No PC que prepara o pacote

Precisa do projeto completo (o `prepare` treina com scikit-learn). Da raiz do repositório:

```
pip install -e .                    # pyruleanalyzer + sklearn/pandas/numpy
```

## 3.2 No PC ligado à placa

Basta Python 3.9+ e a pasta `examples/` do repositório (no mínimo `arduino_stream_test.py` e `arduino_hardware_test.py`, que ele importa):

```
pip install numpy pandas pyserial matplotlib
# matplotlib é opcional: só gera o gráfico comparison.png
```

Para gravar a placa há duas opções:

- **arduino-cli** (automático, recomendado): o script compila, mede a Flash/SRAM reais e grava cada variante sozinho.
- **Arduino IDE** (manual): com `--manual-upload` o script pausa e pede para você gravar cada sketch e apertar Enter. Não há medição de memória.

```
# Windows (ou baixe em https://arduino.github.io/arduino-cli/)
winget install ArduinoSA.CLI

arduino-cli core update-index
arduino-cli core install arduino:avr     # Uno / Nano / Mega / Leonardo
arduino-cli core install esp32:esp32     # ESP32 (requer a URL do board
                                         # manager da Espressif)
arduino-cli board list                   # confere a placa e a porta
```

**Drivers:** clones de Nano e placas ESP32 costumam usar os chips CH340 ou CP2102; no Windows instale o driver correspondente se a porta não aparecer.

## 3.3 Placas suportadas

O `--board` do `prepare` define o FQBN gravado no pacote e os limites de memória usados na estimativa. No `run` ele pode ser trocado por `--board` ou `--fqbn`.

| --board | FQBN | Observação |
|---|---|---|
| `uno` | `arduino:avr:uno` | 2 KB de SRAM: bom para modelos pequenos |
| `nano` | `arduino:avr:nano:cpu=atmega328` | igual ao Uno |
| `nano_old` | `arduino:avr:nano:cpu=atmega328old` | Nano com bootloader antigo (upload falha com nano) |
| `mega` | `arduino:avr:mega` | 8 KB de SRAM: recomendado para bases com muitas features |
| `leonardo` | `arduino:avr:leonardo` | USB nativo (o sketch espera a serial abrir) |
| `esp32` | `esp32:esp32:esp32` | double real de 8 bytes, folga de memória |

Cada linha `D,...` é guardada inteira num buffer na SRAM. Com muitas features (linhas acima de ~600 bytes) o `prepare` avisa: prefira um Mega ou um ESP32.

## 4. Passo 1 — preparar o pacote (`prepare`)

Teste rápido com uma base embutida (iris, wine, breast_cancer ou digits):

```
python examples/arduino_stream_test.py prepare \
    --dataset iris --board uno --name iris_dt
```

Base real com registro temporal (os episódios seguem as transições normal → anomalia de verdade da gravação):

```
python examples/arduino_stream_test.py prepare \
    --train-csv data/swat/train.csv --test-csv data/swat/test.csv \
    --target "Normal/Attack" --time-col Timestamp --drop-cols Timestamp \
    --normal-class Normal --model "Random Forest" --n-estimators 10 \
    --max-depth 6 --board mega --name swat_rf
```

Reaproveitando um modelo já treinado e refinado (um `PyRuleAnalyzer` salvo com pickle). As colunas dos CSVs precisam ser as mesmas do treino:

```
python examples/arduino_stream_test.py prepare --model-pkl files/model.pkl \
    --train-csv ... --test-csv ... --target ... --name meu_modelo
```

### 4.1 Opções principais

| Opção | Padrão | Para que serve |
|---|---|---|
| `--dataset` | iris | Base embutida, quando nenhum CSV é dado. |
| `--csv` | — | Um único CSV, dividido 70/30. |
| `--train-csv / --test-csv` | — | Divisão própria. O cenário usa **só** as linhas de teste. |
| `--target` | última coluna | Coluna do rótulo. |
| `--drop-cols` | — | Colunas a ignorar, separadas por vírgula. |
| `--time-col` | — | Coluna de tempo do teste: episódios cortados nas transições reais. Sem ela, linhas normais são emendadas a linhas de uma classe anômala (em rodízio). |
| `--model` | Decision Tree | Decision Tree, Random Forest ou Gradient Boosting Decision Trees. |
| `--n-estimators / --max-depth` | — / 6 | Tamanho do modelo (impacta direto Flash e tempo). |
| `--model-pkl` | — | Modelo pronto em vez de treinar. |
| `--normal-class` | a mais frequente | Rótulo (ou código) da operação normal. |
| `--episodes` | 20 | Quantos episódios no cenário. |
| `--normal-rows / --anomaly-rows` | 30 / 30 | Linhas antes e a partir do início da anomalia. |
| `--confirm` | 3 | Anormais seguidas que disparam o alarme. |
| `--period-ms` | 100 | Uma amostra a cada N ms. |
| `--board / --baud` | uno / 115200 | Placa alvo e velocidade. |
| `--output-dir / --name` | files/stream / stream_model | Onde o pacote é escrito: `<output-dir>/<name>`. |

### 4.2 O que sai

No final o `prepare` imprime o comportamento esperado de cada variante, por exemplo:

```
[5/5] Expected behaviour on the board (oracle)
  before: detects 6/6 episodes, 0 false alarms, mean delay 3.00 samples
  after : detects 6/6 episodes, 0 false alarms, mean delay 3.00 samples

  package: files/stream/iris_dt
  send this folder to whoever has the board (LEIAME.md).

files/stream/iris_dt/
|-- LEIAME.md              instruções para quem tem a placa
|-- manifest.json          modelo, classes, episódios e oráculo
|-- scenario.csv           cada amostra a enviar + classe esperada
`-- sketches/
    |-- iris_dt_before/iris_dt_before.ino
    `-- iris_dt_after/iris_dt_after.ino
```

**Envie a pasta inteira.** Não edite os sketches nem o `scenario.csv`: o handshake confere o id do modelo gravado na placa e o teste aborta se o sketch errado estiver nela.

## 5. Como o cenário funciona

Um episódio é “a planta funciona normalmente e então começa uma anomalia”. A placa mantém um *streak* de classificações não normais seguidas; quando ele chega a `--confirm` o alarme dispara.

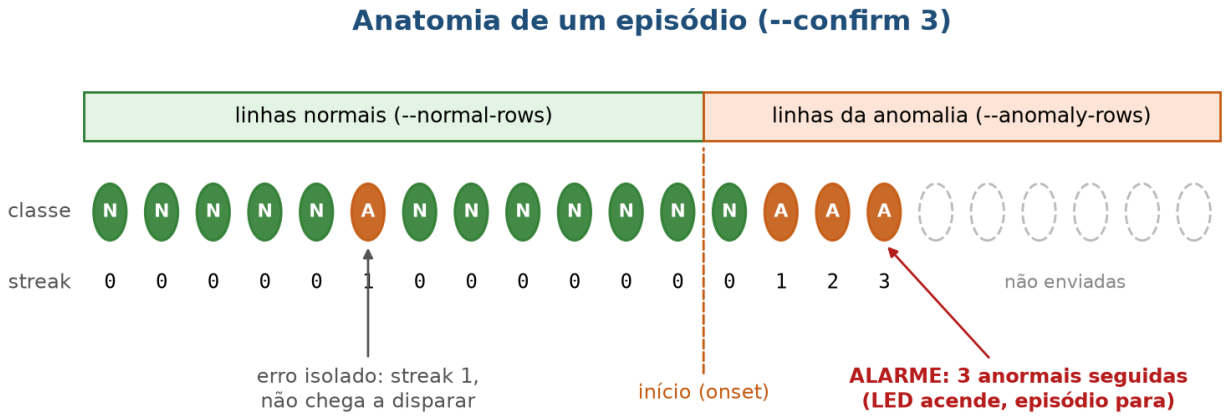



Figura 2 — N = classe normal, A = qualquer outra classe. O episódio termina no alarme; aqui a primeira linha anômala ainda saiu normal, então o atraso de detecção é de 4 amostras.

- **Detecção:** o primeiro alarme a partir do início da anomalia encerra o episódio; o atraso é contado em amostras e em milissegundos.
- **Falso alarme:** um alarme *antes* do início. É registrado, o alarme é zerado (comando `X`) e a fase normal continua.
- **Perda (miss):** o episódio chega à última linha de anomalia sem alarme.

## 6. Passo 2 — ensaio sem placa (`--port sim`)

Antes de mandar o pacote, rode-o no seu PC com uma placa virtual que responde a classe esperada. Isso testa o protocolo, o pacote e o script — **não** o código C (para isso use o host-check da seção 11).

```
python examples/arduino_stream_test.py run \
    --package files/stream/iris_dt --port sim
```

```
[1/2] Variant "before"
  handshake: H,before,1ffef3a13c,4,3,0,0,3
    episode   0: alarm at +3 samples
    ...
[2/2] Variant "after"
  handshake: H,after,271d725b56,4,3,0,0,3
    ...
**Result: PASS** (every sample answered, 100% class parity, alarms where
the oracle says)
```

O relatório de um ensaio traz a marca *Loopback dry run*; tempos de inferência e memória aparecem como 0 ou n/a.

# 7. Passo 3 — rodar na placa (`run`)

## 7.1 Antes de começar

- Conecte a placa e descubra a porta: `arduino-cli board list` (ou Ferramentas → Porta na IDE). No Windows é `COMx`; no Linux, `/dev/ttyACM0` ou `/dev/ttyUSB0`; no macOS, `/dev/cu.usbmodem…`.
- **Feche o Monitor Serial da IDE**: a porta é exclusiva e o upload/handshake falham se ela estiver aberta.
- Rode da raiz do repositório (ou de onde o `examples/` estiver).

## 7.2 Com arduino-cli (automático)

```
python examples/arduino_stream_test.py run --package CAMINHO/iris_dt --port COM5
```

Se `--port` for omitido e o arduino-cli estiver instalado, o script tenta detectar a porta da placa sozinho.

## 7.3 Com a Arduino IDE (manual)

```
python examples/arduino_stream_test.py run --package CAMINHO/iris_dt \
    --port COM5 --manual-upload
```

O script mostra o caminho de cada `.ino` (primeiro `sketches/<nome>_before`, depois `sketches/<nome>_after`). Abra-o na IDE, selecione a placa e a porta, grave, **feche o Monitor Serial** e aperte Enter no terminal.

## 7.4 O que acontece em cada variante

- Compila e grava o sketch (lendo a Flash e a SRAM reais da saída do arduino-cli).
- Abre a serial e espera `--boot-delay` s (placas AVR reiniciam ao abrir a porta).
- Handshake `H`: confere variante e id do modelo; aborta se não baterem.
- Para cada episódio: `X` (zera o alarme) e uma linha `D` por tick, esperando a resposta `C`. Respostas inválidas são reenviadas até `--retries` vezes.

## 7.5 Opções do run

| Opção | Padrão | Para que serve |
|---|---|---|
| `--package` | obrigatório | Pasta gerada pelo prepare. |
| `--port` | auto | `COM5`, `/dev/ttyACM0` ou `sim`. |
| `--board / --fqbn` | do pacote | Troca a placa alvo. |

| Opção | Padrão | Para que serve |
|---|---|---|
| `--baud` | do pacote | Precisa bater com o sketch. |
| `--arduino-cli` | arduino-cli | Caminho do executável. |
| `--manual-upload` | — | Pausa para gravar pela IDE. |
| `--skip-upload` | — | Não grava; use junto com `--variants` para repetir uma variante que já está na placa. |
| `--variants` | before after | Quais variantes rodar. |
| `--period-ms` | do pacote | Muda a taxa de amostragem. |
| `--boot-delay` | 2.5 s | Espera após abrir a porta. |
| `--timeout / --retries` | 3.0 s / 2 | Tempo máximo por resposta e reenvios. |

Exemplo: repetir só a variante **after**, que já está gravada:

```
python examples/arduino_stream_test.py run --package CAMINHO/iris_dt \
    --port COM5 --skip-upload --variants after
```

# 8. Passo 4 — resultados e relatório

Cada execução cria `results/<placa>_<data>_<hora>/` dentro do pacote:

| Arquivo | Conteúdo |
|---|---|
| `before_samples.csv` | Cada amostra enviada: classe esperada, classe da placa, rótulo real, µs de inferência, streak, alarme, RTT e perda de prazo. |
| `after_samples.csv` | Idem para a variante after. |
| `before.json / after.json` | Métricas, resultado de cada episódio e erros de comunicação. |
| `comparison.md / .json` | Tabela before × after e o veredito. |
| `comparison.png` | Atraso de detecção por episódio (perdas marcadas com x). |
| `run.json` | Placa, FQBN, porta, host e taxa usados. |

**Quem rodou na placa deve devolver a pasta `results/` inteira.** Para reimprimir a comparação (da última execução, ou de uma específica):

```
python examples/arduino_stream_test.py report --package CAMINHO/iris_dt
python examples/arduino_stream_test.py report --package CAMINHO/iris_dt \
    --run uno_20261104_151206
```

## 8.1 Lendo a tabela

| Métrica | Significado |
|---|---|
| Rules | Número de regras do modelo (antes × depois do refinamento). |
| Flash / SRAM measured | Uso real, lido do arduino-cli (n/a com upload manual). |
| Inference mean / p95 (µs) | Tempo de `pyra_predict()` medido com `micros()` na placa. |
| Class parity board vs Python | Amostras em que a placa deu a mesma classe que o Python. Deve ser 100%. |
| Accuracy vs true label | Acerto contra o rótulo real da linha. |
| Episodes detected / delay | Detecções e atraso médio/máximo (amostras e ms). |
| False alarms (/1000 normal) | Alarmes antes do início da anomalia. |

| Métrica | Significado |
|---|---|
| Alarm parity with oracle | Episódios em que a placa alarmou exatamente onde o oráculo previa. |
| Deadline misses | Respostas que chegaram depois do próximo tick: inferência mais lenta que a taxa dos sensores. |
| Communication errors | Timeouts, checksum ruim, respostas inválidas. |

**Critério de aprovação (PASS):** para cada variante, (1) todas as amostras respondidas, (2) 100% de paridade de classe e (3) alarmes em todos os episódios exatamente onde o oráculo diz. O processo sai com código 0 (PASS) ou 1 (FAIL), pronto para uso em CI. Acurácia, detecção e falsos alarmes são os números a *comparar* entre before e after, não critérios de aprovação.

## 9. Protocolo serial

Texto, uma mensagem por linha terminada em `\n`. O PC só envia a próxima amostra depois da resposta, então o buffer de 64 bytes do Uno nunca transborda.

| Direção | Mensagem |
|---|---|
| PC → placa | `H` |
| placa → PC | `H,<variante>,<model_id>,<n_feat>,<n_classes>,<n_trees>,<normal>,<confirm>` |
| PC → placa | `X` (zera alarme e streak) → `X,ok` |
| PC → placa | `D,<seq>,f0,...,fN-1*CS` (CS = XOR dos bytes antes do *, 2 dígitos hex) |
| placa → PC | `C,<seq>,<classe>,<inferencia_us>,<streak>,<alarme 0/1>` |
| placa → PC | `E,<motivo>` (checksum ruim, nº de campos errado, linha longa demais…) |

Ao ligar, o sketch imprime `PYRA-STREAM-READY`. Isso permite testar à mão no Monitor Serial (115200, fim de linha “Nova linha”): envie `H` e veja a resposta — mas feche o monitor antes de rodar o script.

## 10. Problemas comuns

| Mensagem / sintoma | Causa provável e solução |
|---|---|
| `No handshake from the board` | Porta errada, Monitor Serial aberto, baud diferente ou sketch de streaming não gravado. Confira com `arduino-cli board list`. Em placas lentas para reiniciar, aumente `--boot-delay`. |
| `The board runs "before" model ..., expected "after"` | O sketch errado está na placa (comum com `--manual-upload` ou `--skip-upload`). Grave `sketches/<nome>_<variante>`. |
| `arduino-cli not found` | Instale o arduino-cli, passe `--arduino-cli C:\caminho\arduino-cli.exe` ou use `--manual-upload`. |
| `Compilation ... failed` | Normalmente o modelo não cabe: reduza `--max-depth` / `--n-estimators` no prepare ou use Mega/ESP32. Confirme que o core da placa está instalado. |
| `Upload failed` | Monitor Serial aberto, porta errada ou Nano antigo (use `--board nano_old`). |
| `Pass --port ...` | A porta não foi detectada: informe `--port` explicitamente. |
| Paridade de classe abaixo de 100% | Em AVR, `double` tem 4 bytes: amostras quase exatamente sobre um limiar podem divergir. Se todas as divergências estão na fronteira, não é bug de lógica; no ESP32 o efeito some. Rode o host-check (seção 11) para separar exportador × placa. |

| Mensagem / sintoma | Causa provável e solução |
|---|---|
| Muitos deadline misses | A inferência é mais lenta que `--period-ms`: aumente o período ou diminua o modelo. Não invalida o teste, mas mostra que a placa não acompanha aquela taxa. |
| `E,line_too_long` / `E,bad_feature_count` | Muitas features para o buffer da placa, ou CSV com colunas diferentes do treino. Use Mega/ESP32 e confira `--drop-cols`. |

## 11. Os outros testes com Arduino do repositório

O teste de streaming fecha um ciclo de três scripts em `examples/`. Vale rodar os outros dois antes, porque eles isolam problemas mais cedo:

| Script | Precisa de placa? | O que verifica |
|---|---|---|
| `arduino_hardware_test.py` | Opcional | Teste HIL vetor a vetor: paridade C × Python, latência e memória. Com `--host-check --no-hardware` compila o mesmo C com g++ no PC (ideal para CI). Tutorial próprio: `tutorial_hil_arduino_pt.pdf`. |
| `arduino_sensor_simulation.py` | Não | Roda o sketch de produção (setup/loop) no PC com g++ e um shim de Arduino, alimentado por sinal sintético ou pelas linhas reais (ADC ou serial). |
| `arduino_stream_test.py` | Sim (ou `sim`) | Este tutorial: streaming na placa real, before × after, com alarme. |

```
# só no PC, sem placa: valida o C gerado contra o Python
python examples/arduino_hardware_test.py --dataset iris --host-check --no-hardware

# simulação contínua do sketch de produção no PC
python examples/arduino_sensor_simulation.py
```

Ordem sugerida: host-check → simulação → `prepare` → `run --port sim` → `run` na placa → `report`.